Disease-Only Plant Disease Classification Across Host Plants

A constraint-aware technical report on model selection, augmentation strategy, ablations, and deployment-oriented trade-offs

Taron Babayan

Prepared for the plant disease challenge requiring disease prediction independent of host plant identity

Abstract

This report studies plant disease classification under a deliberately harder label formulation: the model must predict the disease itself while ignoring host-plant identity. That requirement changes the problem in a meaningful way. A standard crop-and-disease classifier can partially solve the task by exploiting plant-specific appearance, but a disease-only system must instead learn lesion texture, shape, discoloration patterns, and other symptom-level cues that transfer across crops. To align the dataset with that objective, folder names containing both plant species and disease names were remapped into a disease-only taxonomy. The resulting label space contained 39 classes across 7,783 official training images and 390 balanced official validation images.

The experimental program was designed to be both practical and interpretable. Because the challenge explicitly rewards accuracy, efficient methodology, authenticity, and zero-cost tooling, I prioritized compact pretrained models that can be trained on free Colab hardware and later hosted locally without third-party inference APIs. The work began with a controlled ResNet50 + MLP transfer-learning baseline, then expanded to a lightweight MobileNet comparison, a stronger YOLO11s-cls line, an exploratory CBAM-augmented YOLO branch, a focal-loss variant, and a two-stage Swin Tiny transformer baseline. Experiment tracking was done with Weights & Biases, and the most reproducible YOLO runs were saved with public W&B URLs.

The strongest deployment-oriented result came from a clean YOLO11s-cls configuration trained at 320 × 320 with moderate augmentation. In the final W&B-tracked run, it reached 85.19% top-1 accuracy on the internal dev_sub split and 78.46% top-1 accuracy with macro mAP 0.8668 on the official validation split. A separate earlier clean YOLO run reached 79.74% official accuracy and macro mAP 0.8704, which suggests that the method is not winning by a single lucky seed alone but is consistently competitive in the upper-70% range. ResNet50 remained a useful baseline but plateaued near 71.54% official accuracy. Swin Tiny performed strongly once the final stage was unfrozen, but it was much heavier. Focal loss improved internal fit but did not improve final robustness. CBAM increased model complexity slightly yet reduced official validation performance.

The main conclusion is that this challenge is governed less by flashy architectural add-ons and more by whether the model learns transferable symptom features under careful augmentation, disciplined evaluation, and meaningful backbone adaptation. In this setting, a compact YOLO classifier offered the best balance of accuracy, parameter efficiency, local deployability, and experimental clarity.

Keywords: plant disease classification, disease-only labeling, transfer learning, YOLO classification, Swin Transformer, augmentation, class imbalance, W&B

1. Challenge framing and design constraints

The challenge definition shaped the entire project. The input is a plant or plant-leaf image, but the expected output is the disease only. In other words, “tomato late blight” and “potato late blight” are not two labels in

the final system; both must collapse into “late blight”. This is not a cosmetic relabeling step. It removes an important shortcut that many models use in ordinary plant-disease datasets, namely the correlation between a disease and the host species on which it most often appears.

The submission constraints were equally important. The model had to be hostable on local hardware, trained with zero-cost tools, and built without relying on ready-made specialized plant-disease models or third-party inference APIs. The report and code were also graded for efficiency, clarity, and authenticity. That combination of requirements made it unreasonable to chase very large architectures or opaque, over-tuned search procedures. I instead treated the task as a careful engineering problem: start with a credible baseline, stress-test the assumptions, and move only when there is a clear reason to move.

2. Dataset reformulation, splitting logic, and leakage control

The official dataset contained approximately 8k images across train and validation directories. In the runs for which I have direct logs, the official training set contained 7,783 images and the official validation set contained 390 images, with 39 disease labels in the disease-only formulation. The official validation set was balanced at 10 images per class, which made it especially useful for per-class diagnostics rather than just aggregate accuracy.

To make the task match the challenge statement, I parsed folder names, separated plant tokens from disease tokens, and rewrote the target space so that each image contributed only its disease label. This reformulation was the central methodological decision of the project. It intentionally made the problem harder, because the model had to become less sensitive to crop identity and more sensitive to symptom morphology.

For the YOLO and Swin experiments, I also created a clean three-way split: train_sub, dev_sub, and official_val. In the fully W&B-tracked clean YOLO run, this produced 6,615 images in train_sub, 1,168 in dev_sub, and 390 in official_val, with all 39 classes present in each split and no file overlap across splits. The label distribution remained imbalanced: in train_sub the mean class count was 169.6, the minimum was 76, and the maximum was 736. That imbalance later motivated class weighting, oversampling experiments, and focal-loss testing.

Later experiments also became stricter about leakage control. In one of the archived YOLO ablations, duplicate handling within splits was enforced and training images matching official-validation hashes were removed. That made the result slightly more conservative, but it also made the conclusion more trustworthy. Because the final goal is generalization across host plants and image conditions, it is better to lose a little headline accuracy than to keep a protocol that could quietly inflate it.

Table 1. Task constraints and how they influenced the experimental design.

Constraint	Implication for modeling	Design response in this project
Disease-only output	Model cannot rely on plant identity as the main cue	Labels were remapped from plant–disease combinations to disease-only classes
On-premise ready	Final model should remain compact and easy to serve	Preference was given to ResNet-, YOLO-, and Swin-scale backbones rather than very large vision models
Zero-cost training	Training budget and search budget are limited	Experiments were staged and interpretable; Optuna was considered but not made the center of the study

Efficiency matters in grading	A slightly stronger but far heavier model is not automatically better	Parameter count, weight size, and inference practicality were part of model selection
W&B experiment tracking	Reported results should be traceable	Public W&B runs were kept for the main reproducible baselines

3. Experimental protocol

3.1 Preprocessing and resize policy

Image size and resize policy were treated as model-dependent rather than fixed by habit. The ResNet50 baseline used 320 × 320 inputs with aspect-ratio-preserving padding. I preferred padding over direct distortion because the dataset contains highly variable aspect ratios, including wide and high-resolution images, and I did not want a baseline result to be artificially penalized by unnecessary geometric warping. The same general logic carried into later experiments: preserve lesion structure when possible, and only use stronger geometric changes when there is a clear robustness benefit.

YOLO11s-cls was mainly trained at 320 × 320 for the compact baseline, while the focal-loss variant was intentionally raised to 640 × 640. That higher-resolution focal run was not chosen arbitrarily; it was a deliberate test of whether more detail plus hard-example emphasis would help subtle disease boundaries. Swin Tiny was run at 384 × 384 because that resolution fits the architecture more naturally.

3.2 Augmentation strategy and why each augmentation was chosen

Augmentation was one of the most carefully controlled parts of the project. The goal was not to maximize synthetic variety at any cost, but to enlarge the training distribution without altering the disease semantics. In practice this meant choosing transformations that mimic plausible camera variation, framing variation, and illumination variation while avoiding transformations that can literally rewrite the shape of a lesion or create biologically implausible leaves.

For the ResNet50 + MLP notebook, the augmentation block used horizontal flipping, no vertical flipping, rotation factor 0.08, zoom factor 0.10, translation height and width 0.08, and mild brightness/contrast changes of 0.10 each. This was a deliberately conservative recipe. Horizontal flipping is usually safe because a diseased leaf remains the same disease when mirrored left-to-right. Vertical flipping was disabled because upside-down plant imagery is less common in real capture settings and can introduce unrealistic orientation patterns. Small rotation and translation improve viewpoint robustness, while moderate zoom helps when disease occupies only part of the leaf. Brightness and contrast shifts were kept mild so the model would learn stable color relations rather than latch onto one lighting condition.

The Swin Tiny training kept the same philosophy but expressed it in the torchvision/timm pipeline: horizontal flip, rotation of roughly 29 degrees, translation 0.08, scale range 0.90–1.10, brightness 0.10, and contrast 0.10. Again, the reasoning was to tolerate moderate acquisition variability without destroying the local symptom structure that the model needs to separate visually similar diseases.

The YOLO family used the Ultralytics classification pipeline, so the augmentation knobs were different. The clean W&B-tracked run used hsv_h = 0.015, hsv_s = 0.4, hsv_v = 0.4, fliplr = 0.5, flipud = 0.0, translate = 0.1, scale = 0.5, mosaic = 1.0, erasing = 0.0, and auto_augment = randaugment. This recipe may look more aggressive at first glance, but it still follows the same principle. Horizontal flipping remains safe. Vertical flipping was still disabled. HSV jitter was used because disease cues often involve discoloration, but the model must not assume that every rust or mildew image is captured under identical lighting. Translation and

scale matter because disease patches can occupy different portions of the frame. Mosaic, which is part of the Ultralytics pipeline, acts as a strong regularizer and encourages the backbone to learn more stable local features rather than memorizing one canonical layout per class. I deliberately left erasing off in the clean run because random removal of image content can be risky when the diagnostic lesion itself may be small.

The focal-loss YOLO run intentionally simplified augmentation in one respect: auto_augment was turned off. This was important methodologically. Since the purpose of that run was to test whether focal loss itself changed behavior on hard, imbalanced classes, it made sense to reduce the number of moving parts around it rather than simultaneously stacking stronger policy-based augmentation and a different loss.

3.3 Metrics, experiment tracking, and what counted as a trustworthy improvement

Because the challenge score is driven primarily by average precision on a hidden test set, I did not rely on plain accuracy alone. Whenever the evaluation pipeline allowed it, I tracked top-1 accuracy, macro F1, weighted F1, ROC AUC in a one-vs-rest setting, and macro average precision. This made the interpretation much more honest. Accuracy can look comfortable even when minority classes or harder disease families are underperforming.

Experiment tracking was done with Weights & Biases. The project kept public W&B runs for the main ResNet50 baseline, the final clean YOLO baseline, and the CBAM branch. That was important not only for convenience but also for submission quality: the challenge explicitly asks for experiment tracking, and a public W&B URL is a clean way to show training curves, run configuration, and model selection history.

Table 2. Augmentation choices and the reasoning behind them.

Experiment family	Key augmentation settings	Why they were chosen	Why stronger alternatives were avoided
ResNet50 / MobileNet	fliplr; rot=0.08; zoom=0.10; translate=0.08; brightness/contrast=0.10; no vertical flip	Robustness to framing and lighting while preserving disease geometry	Large warps risk changing lesion appearance and hurting already-limited baselines
Swin Tiny	fliplr; ~29° rotation; translate=0.08; scale 0.90–1.10; brightness/contrast=0.10	Matches transformer baseline with moderate invariance and no unrealistic distortion	Heavier augmentation would make it hard to tell whether gains came from the architecture or the policy
YOLO clean baseline	HSV jitter; fliplr=0.5; translate=0.1; scale=0.5; mosaic=1.0; auto_augment=randaugment; erasing off	Local-feature learning under varied crop, scale, and illumination conditions	Erasing could hide the lesion itself; vertical flips were not realistic enough to justify
YOLO focal-loss run	Same moderate color/flip policy, but auto_augment disabled	Isolate the effect of focal loss and high resolution	Too many simultaneous changes would blur the interpretation

4. Model progression and the logic behind each transition

4.1 ResNet50 + MLP as the controlled starting point

I began with an ImageNet-pretrained ResNet50, not because it is the newest backbone available, but because it is a reliable transfer-learning baseline with enough capacity to solve a meaningful share of the task while

remaining interpretable. The classifier head was a small MLP with hidden sizes 512 and 128, batch normalization, and dropout 0.30. Stage 1 trained only the head for 20 epochs with an initial learning rate of 1e-3. Stage 2 then unfroze the last 30 backbone layers and fine-tuned for 10 epochs at 1e-5. Class weights were enabled to counter the substantial label imbalance.

This baseline did what a good baseline should do: it behaved predictably, showed that the disease-only problem was learnable, and exposed where the difficulty really lived. In the logged run, stage 1 achieved its best validation-loss checkpoint at 1.1111 with validation accuracy 68.72%, while the best observed stage-1 validation accuracy peaked slightly higher at 70.51%. Stage 2 improved the best validation-loss checkpoint to 1.0315, and the final official-validation classification report yielded 71.54% accuracy with macro F1 0.7136. The model clearly learned something real, but it also made it obvious that disease-only recognition under mixed backgrounds and cross-crop variation would need a stronger spatial representation.

4.2 MobileNetV2 as an efficiency stress-test

After ResNet50, I deliberately checked a lighter architecture rather than immediately moving to something heavier. The reason was tied directly to the grading rubric: methodology rewards efficiency, not just raw accuracy. A smaller backbone would have been attractive if it stayed competitive.

In practice, MobileNet behaved like a useful negative result. Earlier experimental notes placed it at roughly 60% validation accuracy, clearly below the stronger baselines. That outcome mattered. It showed that the disease-only formulation punishes overly compact representations more than an easier plant-and-disease formulation might. Once plant identity is removed as a shortcut, the model has to separate quite subtle texture and lesion differences, and MobileNet was simply too weak to become the main line of work.

4.3 Why the project moved toward YOLO11s-cls

The shift from ResNet50 to YOLO11s-cls was not a trend-following move. It came from the visual nature of the problem. Many diseases in the dataset are expressed through relatively localized evidence: small spots, clusters of lesions, edge discoloration, patchy mildew, narrow streaks, or rust-like speckling. Even in a classification setting, a backbone that is strong at extracting multi-scale local structure can have an advantage.

The clean YOLO11s-cls baseline confirmed that reasoning. In one earlier archived run, the model reached 83.90% accuracy with macro F1 0.8136 and macro mAP 0.8730 on dev_sub, then 79.74% accuracy with macro F1 0.7992 and macro mAP 0.8704 on official validation. A later W&B-tracked clean run reached 85.19% on dev_sub and 78.46% on official validation with macro mAP 0.8668. The exact number moved a little from run to run, but the conclusion did not: YOLO gave a clear step up over the ResNet baseline while staying compact, with about 5.49M parameters and stripped best weights of only 11.1MB. The clean W&B-tracked run also completed 23 epochs in about 1.39 hours, which was a strong efficiency result for a zero-cost Colab workflow.

4.4 Head-only adaptation and staged fine-tuning

Once YOLO became the strongest family, the next question was not whether YOLO was good, but why it was good. I specifically wanted to know whether its gains came mostly from the pretrained representation or from the backbone adapting to this disease-only domain. That led to a head-only ablation. Archived experiment notes from an earlier strict run placed this setting around 52.6% top-1 accuracy with macro mAP around 0.627. Even allowing for the fact that the number comes from archived notes rather than the final reproducibility package, the conclusion is clear enough: the backbone cannot remain frozen if the goal is real disease-only generalization.

That observation also explains why staged fine-tuning became attractive. In archived notes from a later YOLO branch with oversampling and staged unfreezing, stage 1 reached roughly 75.2% validation top-1 accuracy and stage 2 climbed to about 83.5% with top-5 around 98.2%. The important part is not the exact archived number; it is the pattern. Letting the head stabilize first and then unfreezing more of the model produced a materially stronger discriminator. That is consistent with the rest of the project: backbone adaptation mattered more than I initially expected.

4.5 Focal loss: a well-motivated idea that did not fully transfer

Focal loss was not introduced as a random trick. It was chosen because the dataset combines class imbalance with visually ambiguous categories. Under those conditions, it is reasonable to ask whether the model should spend more learning capacity on hard examples instead of continuing to accumulate confidence on easy, dominant classes.

The focal-loss YOLO notebook used 640 × 640 inputs, 30 epochs, batch size 32, patience 7, Adam with lr0 = 1e-4, cosine learning-rate decay, and focal gamma 2.0. The result was mixed in a very informative way. In archived metrics from that run, dev_sub reached 84.33% top-1 accuracy with macro F1 0.8257 and macro mAP 0.8921, which looked excellent. But official-validation performance dropped to 73.85% top-1 accuracy with macro F1 about 0.7407 and macro mAP 0.8279. In other words, focal loss improved internal fit without giving the kind of final generalization gain I needed. That is exactly the type of result worth keeping in a serious report: a method can be theoretically sensible and still fail the external test that matters.

4.6 CBAM as an exploratory attention branch

The CBAM branch was motivated by a different question. If diseases are often localized, could attention help the network focus more strongly on the visually informative regions? The custom YOLO11s-CBAM model inserted CBAM blocks into the backbone with kernel size 7 and reduction ratio 16. Relative to the clean YOLO baseline, this increased the model only slightly, from about 5.49M to 5.57M parameters and from 12.2 to 12.3 GFLOPs. In principle that is a reasonable price for potentially better feature selection.

In practice, the branch did not deliver a win. The run logs explicitly recorded that CBAM modules were found in the architecture but the tensor-capture check reported zero captured CBAM tensors, which already made the branch methodologically suspicious. Performance confirmed the concern. The official-validation result was 73.85% top-1 accuracy with macro F1 0.7406 and macro mAP 0.8412, well below the clean YOLO baseline. The dev_sub result, 81.85% accuracy with macro mAP 0.8573, was also lower than the clean baseline. I therefore kept the branch in the report as an informative ablation but not as a headline contribution.

4.7 Swin Tiny as the strongest non-convolutional counterpoint

It would have been too easy to conclude from the early results that the answer was simply “use YOLO”. I wanted a real alternative family in the study, which is why I trained a Swin Tiny transformer as a compact hierarchical transformer baseline. Stage 1 trained only the classifier head: just 29,991 parameters out of 27,549,345 total were trainable. As expected, this was not enough; the best validation accuracy in that stage reached 63.53%.

Stage 2 unfroze the final Swin stage plus the head, increasing the trainable count to 15,398,103. That changed the picture substantially. The best validation accuracy rose to 80.74% and the final official-validation evaluation reached 76.15% top-1 accuracy with top-5 accuracy 97.69%. This was a strong result, and it taught the same lesson as the head-only YOLO ablation: pretraining helps, but partial domain adaptation is essential. Even so, Swin Tiny was much heavier than the selected YOLO model, so it did not become the best deployment-oriented choice.

5. Results

Table 3 summarizes the main experiments. Some entries are fully reproducible from the attached notebooks and logs, while a few earlier supporting ablations are marked as archived-note results because their original notebooks were not part of the final reproducibility package. I kept them in the table anyway because they still help explain the path of the project, but I intentionally distinguish them from the final W&B-tracked runs.

Table 3. Consolidated summary of the main experiments.

Model	Input	Eval split	Top-1 (%)	Macro F1	Macro mAP	Size / params	Interpretation
ResNet50 + MLP (2-stage)	320	Official val	71.54	0.7136	—	24.7M total 1.12M trainable in stage 1	Frozen backbone first, then last 30 layers fine-tuned
MobileNetV2 (lightweight baseline)	320	Validation	60.00	—	—	lightweight	Approximate result from earlier notes; useful efficiency check but too weak
YOLO11s-cls (earlier clean run)	320	Official val	79.74	0.7992	0.8704	5.49M	Earlier archived run from prior report; shows upper-70s consistency
YOLO11s-cls (final W&B-tracked clean run)	320	Official val	78.46	0.7879	0.8668	5.49M 12.0 GFLOPs 11.1MB stripped weights	Final deployment-friendly selection
YOLO11s-cls head-only	320	Held-out	52.60	—	0.6270	5.49M	Archived ablation; frozen backbone was insufficient
YOLO11s-cls + oversampling + staged unfreezing	320	Validation	83.50	—	—	5.49M	Archived notes; stage 1 around 75.2%, stage 2 around 83.5%
YOLO11s-cls + focal loss	640	Official val	73.85	0.7407	0.8279	5.49M	Gamma=2.0, Adam, cosine LR; better internal fit, worse final generalization
YOLO11s-cls + CBAM	320	Official val	73.85	0.7406	0.8412	5.57M 12.3 GFLOPs	Same data flow as clean YOLO, but extra attention blocks

							did not pay off
Swin Tiny (2-stage)	384	Official val	76.15	—	—	27.55M total 15.40M trainable in stage 2	Strong non-convolutional baseline, but heavier than YOLO

Figure 1. Official-validation top-1 accuracy across the main model families.

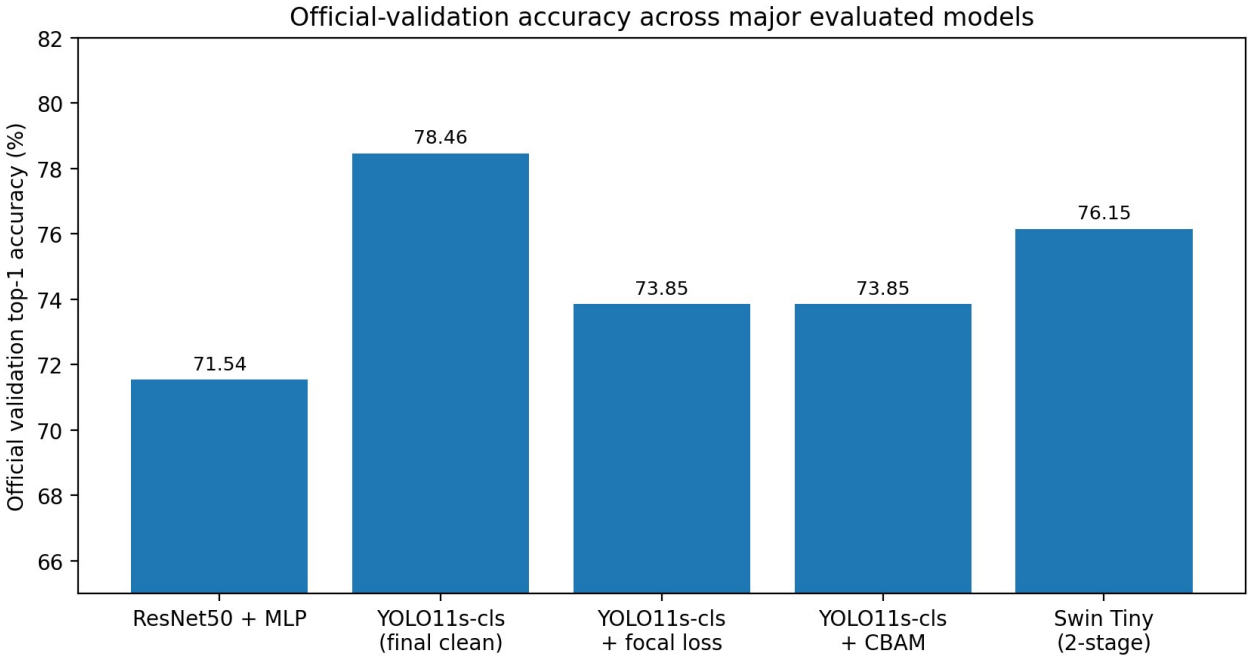
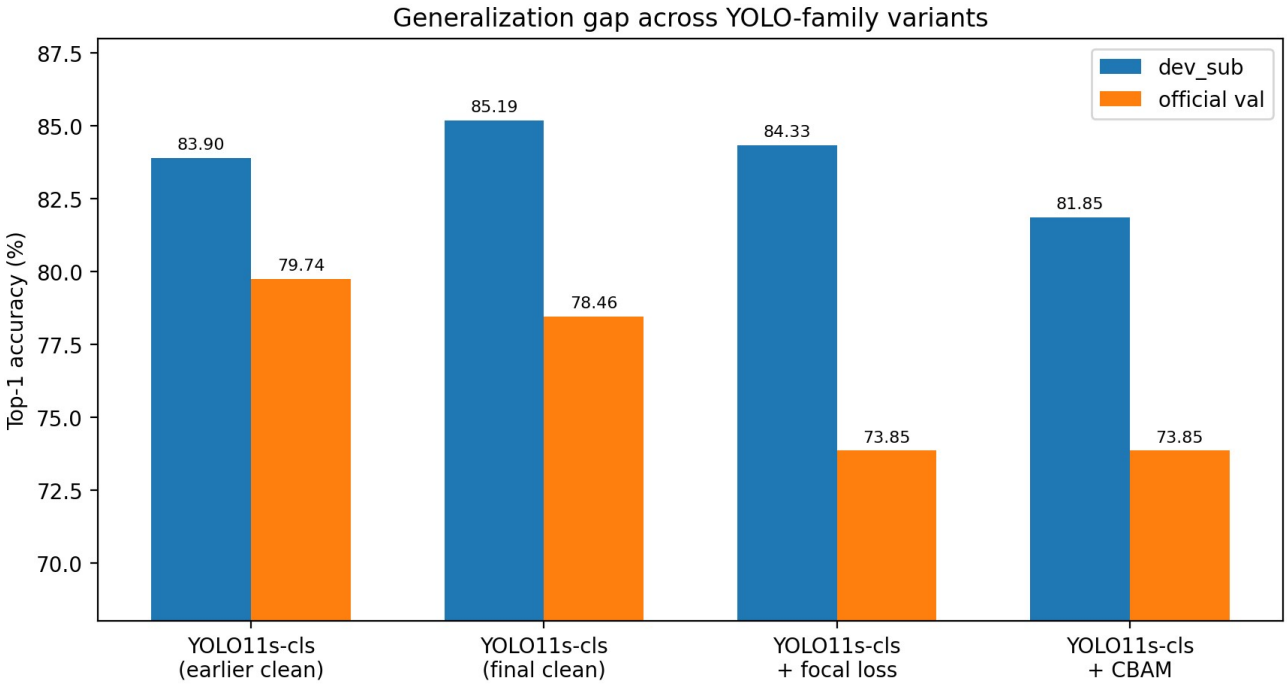


Figure 2. dev_sub versus official-validation accuracy for the YOLO-family variants. The widening gap in the focal-loss and CBAM branches is one reason they were not selected as the final direction.



Two patterns stand out from the results table and figures. First, moving from the ResNet baseline to YOLO produced the clearest practical gain. Even when the exact official-validation number shifted across clean runs, YOLO stayed ahead of ResNet by roughly seven to eight percentage points while remaining smaller and easier to deploy. Second, the strongest internal fit was not always the strongest final model. The focal-loss run is the clearest example: it improved internal mAP but produced a worse official-validation result than the simpler clean baseline.

The archived earlier clean YOLO run at 79.74% official validation and the later W&B-tracked clean run at 78.46% should not be read as a contradiction. They are close enough that they actually strengthen the story. They suggest the method is fairly stable in the upper-70% range rather than relying on one unusually lucky experiment. For a challenge with cross-crop disease-only labels, diverse image conditions, and only around 8k images, that stability matters.

6. Discussion

6.1 What the project learned about this task

The most important lesson from the entire study is that disease-only classification is not just a slightly harder version of ordinary plant classification. It changes which cues matter. Once plant identity stops being a label-level shortcut, the model has to distinguish diseases using texture, lesion layout, edge damage, and color transitions that are often much more subtle than crop-level morphology. This is why backbone adaptation repeatedly proved essential. Head-only training underperformed for both convolutional and transformer families.

The second lesson is that compactness and strong local feature extraction can coexist. The selected YOLO11s-cls model is not a giant architecture, yet it consistently outperformed the heavier baselines I tried. That makes it especially attractive under the challenge rubric, which values efficient methods rather than pure parameter scale.

The third lesson is that not every theoretically sensible modification is worth keeping. Both focal loss and CBAM had good motivations. Focal loss targeted hard examples under imbalance; CBAM targeted localized relevance. But the actual task rewarded the simpler recipe more. This is one reason I prefer to keep those runs in the report as informative failures rather than hide them.

6.2 Error structure and what the remaining mistakes mean

The errors were not random. Across runs, the hardest families were the ones that are genuinely similar in symptom appearance: rust-type labels, leaf-spot variants, septoria-like categories, mildew-type labels, and some mosaic-related cases. The per-class reports from both ResNet and YOLO support that reading. For example, bacterial leaf spot and septoria leaf spot remained weaker than the most visually distinctive diseases, while classes such as bunchy top, black leaf streak, or smut were often much easier once the model had a strong representation.

I do not think these remaining mistakes imply that the model was learning nonsense. A more cautious interpretation is that disease-only separation from a single RGB image can be intrinsically ambiguous for some labels, especially when host identity is deliberately removed. That ambiguity is exactly why the task is interesting.

6.3 Why Optuna was considered, but not made central

Automated hyperparameter search with Optuna was considered throughout the project. It would have been a reasonable next step for learning rate, augmentation magnitude, dropout, warmup behavior, or stage lengths.

I chose not to make it the center of the present study for two reasons. First, zero-cost compute strongly limits how broad a search can be before it turns into noise. Second, I wanted the report to say something interpretable about architecture choice and training dynamics, not only present the result of a search budget.

That said, Optuna remains one of the clearest next improvements. The clean YOLO baseline is already compact and stable, which means a moderate, well-scoped Optuna search could be meaningful instead of chaotic. The most sensible search space would include input size, patience, optimizer selection, augmentation strength, and perhaps whether to keep or disable auto_augment. I would not begin with an enormous search; I would begin with a narrow one around the current strong baseline.

6.4 Why alternative color spaces were considered

I also considered alternative color spaces such as HSV and Lab. This idea came from a practical intuition rather than from wanting to add unnecessary complexity. In plant pathology imagery, disease evidence is often color-sensitive, but raw RGB mixes intensity and chromatic information in a way that is not always ideal. HSV can separate hue from brightness more explicitly, and Lab can make perceptual color differences easier to represent.

I did not ultimately switch the main models away from RGB because modern pretrained backbones already learn rich color-sensitive filters, and I did not have enough controlled evidence to claim that a manual color-space transform would reliably improve this dataset. In other words, the idea was real, but it was not yet better justified than the current baseline. If the project were extended, the cleanest way to test it would be an ablation that keeps everything else fixed and compares RGB, RGB+HSV, and RGB+Lab under the same training schedule.

7. Deployment fit and submission readiness

The final selected direction for deployment is the clean YOLO11s-cls baseline. It is not only strong in accuracy but also better aligned with the rest of the submission requirements. It is compact, locally hostable, trained without third-party inference APIs, and already connected to W&B for reproducible tracking. The stripped best weights were about 11.1MB, and the fused model contains roughly 5.48M parameters. On the logged T4 environment, validation inference was in the low-millisecond range per image.

That makes the model a natural candidate for the required production API. This report does not pretend that the modeling notebook is the API itself, but it does show that the selected architecture is practical for a FastAPI service with Swagger/OpenAPI documentation. In short, the chosen model is not only competitive in validation metrics; it also fits the operational constraints of the challenge much better than the heavier alternatives.

Appendix A lists the public W&B URLs associated with the main tracked experiments. Those runs form the reproducible backbone of the report.

8. Conclusion

This project approached the challenge the way it should be approached: not as a pure leaderboard exercise, but as a constraint-aware modeling problem. The disease-only labeling requirement was treated as fundamental, not as a minor dataset cleanup step. The experiments were then built around that requirement and around the practical submission constraints of local deployability, zero-cost training, and methodological efficiency.

ResNet50 + MLP provided a credible first baseline and showed that the problem was learnable, but it also exposed the ceiling of a straightforward transfer-learning recipe. MobileNet demonstrated that efficiency

alone was not enough. YOLO11s-cls then became the strongest direction because it combined compactness with strong local feature extraction. Head-only and staged-unfreezing ablations clarified that backbone adaptation was essential. Focal loss and CBAM were both well-motivated ideas, but neither improved the final deployment-oriented result. Swin Tiny confirmed that a transformer can work well, but it did so at a heavier cost.

The final conclusion is therefore simple and well supported: for disease-only plant disease classification across host plants, the most effective recipe in this project was a compact pretrained model with strong spatial inductive bias, moderate and biologically sensible augmentation, disciplined attention to split integrity, and enough fine-tuning freedom for the backbone to adapt. That recipe produced a practical, reproducible, on-premise-ready model and a set of experiments that explain not only what worked, but why the project moved from one experiment to the next.

References

Antwi, K., Bennin, K. E., Asiedu, D. K. P., & Tekinerdogan, B. (2024). On the application of image augmentation for plant disease detection: A systematic literature review. Smart Agricultural Technology, 9, 100590.

Kim, H. E., Cosa-Linan, A., Santhanam, N., Jannesari, M., Maros, M. E., & Ganslandt, T. (2022). Transfer learning for medical image classification: A literature review. BMC Medical Imaging, 22, 69.

Kaur, R., Mittal, U., Wadhawan, A., Almogren, A., Singla, J., Bharany, S., Hussen, S., Rehman, A. U., & Al-Huqail, A. A. (2025). YOLO-LeafNet: A robust deep learning framework for multispecies plant disease detection with data augmentation. Scientific Reports, 15, 28513.

Lin, T.-Y., Goyal, P., Girshick, R., He, K., & Dollar, P. (2017). Focal loss for dense object detection. Proceedings of the IEEE International Conference on Computer Vision.

Liu, Z., Lin, Y., Cao, Y., Hu, H., Wei, Y., Zhang, Z., Lin, S., & Guo, B. (2021). Swin Transformer: Hierarchical vision transformer using shifted windows. Proceedings of the IEEE/CVF International Conference on Computer Vision.

Redmon, J., Divvala, S., Girshick, R., & Farhadi, A. (2016). You Only Look Once: Unified, real-time object detection. Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition.

Shoaib, M., Shah, B., El-Sappagh, S., Ali, A., Ullah, A., Alenezi, F., Gechev, T., Hussain, T., & Ali, F. (2023). An advanced deep learning models-based plant disease detection: A review of recent research. Frontiers in Plant Science, 14, 1158933.

Woo, S., Park, J., Lee, J.-Y., & Kweon, I. S. (2018). CBAM: Convolutional Block Attention Module. Proceedings of the European Conference on Computer Vision.

Yang, B., Li, M., Li, F., Wang, Y., Liang, Q., Zhao, R., Li, C., & Wang, J. (2024). A novel plant type, leaf disease and severity identification framework using CNN and transformer with multi-label method. Scientific Reports, 14, 11664.

Appendix A. Public experiment tracking URLs and reproducibility notes

ResNet50 baseline W&B: https://wandb.ai/taron-babyan-yerevan-state-university-ysu/metric-plant-disease/runs/resnet50_disease_20260418_161342_85470d80

Final clean YOLO W&B: https://wandb.ai/taron-babyan-yerevan-state-university-ysu/metric-plant-disease-yolo-cls/runs/0548am3q

CBAM exploratory W&B: https://wandb.ai/taron-babyan-yerevan-state-university-ysu/metric-plant-disease-yolo-cls/runs/njk8zt4n

The most reproducible experiments in this report are the ResNet50 baseline, the final clean YOLO run, the CBAM run, and the Swin notebook. A few supporting figures—especially the head-only YOLO, the oversampling/staged-unfreezing branch, and the lightweight MobileNet baseline—are retained from earlier archived notes and the earlier report draft. They are kept because they clarify the experimental path, but they are identified as archived-note results rather than presented as if they were part of the final reproducibility package.

That distinction is intentional. The report is meant to be honest first and polished second. When a result is fully tied to the attached notebook/log package, I report it as such. When it comes from earlier notes that informed the direction of the project, I still keep it in the narrative, but I label it accordingly.